Building an AI Agent Harness for Conducting Scientific Experiments

Exploring heat-induced aggregation of egg-white proteins

Sina Shahandeh · October 2026

1 Design rationale

In this short project, I have built a complete end-to-end setup of an AI agent interacting with the physical world and conducting a scientific experiment. The idea is to convert a 3D printer into a robotic experimentation platform and have an AI agent autonomously control the movement of a camera installed on its head and the temperature of its bed (Figure 1). This setup allows investigation of heat-induced phenomena while the camera measures color or opacity changes in the materials placed on the bed. Given the temperature range available on the 3D printer, I decided to investigate the kinetics of aggregation of the protein molecules present in egg white. Heat-induced changes in protein structure are an important scientific topic, both in drug design and delivery and in food science.

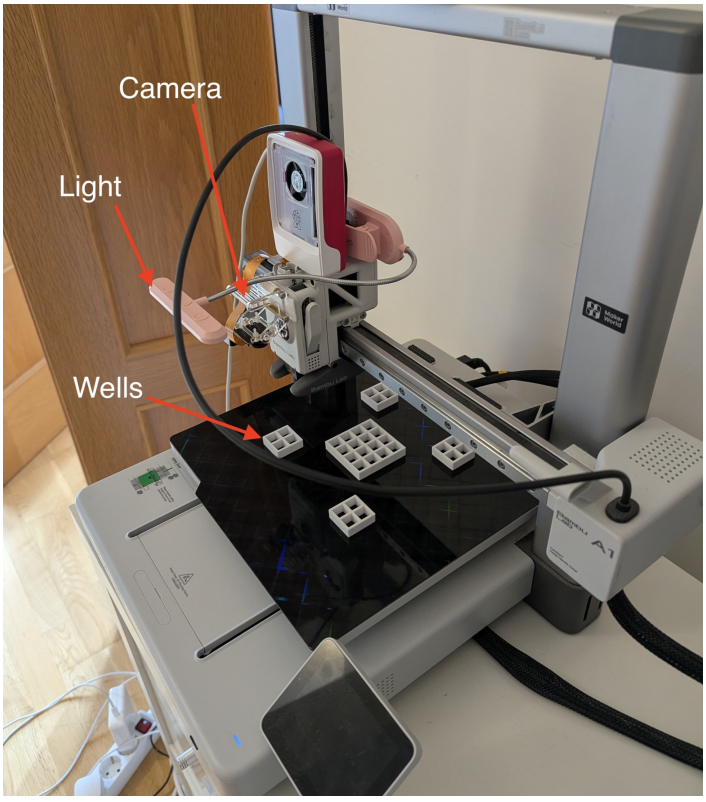


Figure 1. The experimental rig. A Bambu Lab A1 3D printer carries a Raspberry Pi camera on its toolhead. A grid of wells is arranged on the heated bed.

Although current coding agents are able to write scripts to control a 3D printer or a robot, they are not fundamentally "in the loop" during the scientific experiment. Claude Code can write a script to move a

robotic setup, use a camera to take a photo and get feedback from the environment. However, such a system is fragile; it is hard to implement safety checks coming from the hardware, and it is hard to scale. Because it is cumbersome for the AI agent to maintain the low-level drivers for robot control, it limits, through the context window size, how many instruments and how long an experiment an agent can handle. We need abstractions that separate the actual work of conducting the scientific experiment from hardware control or higher-level ideation. Conducting scientific experiments has a hierarchical dynamic that is different from tasks such as coding. Figure 2 shows this hierarchy schematically.

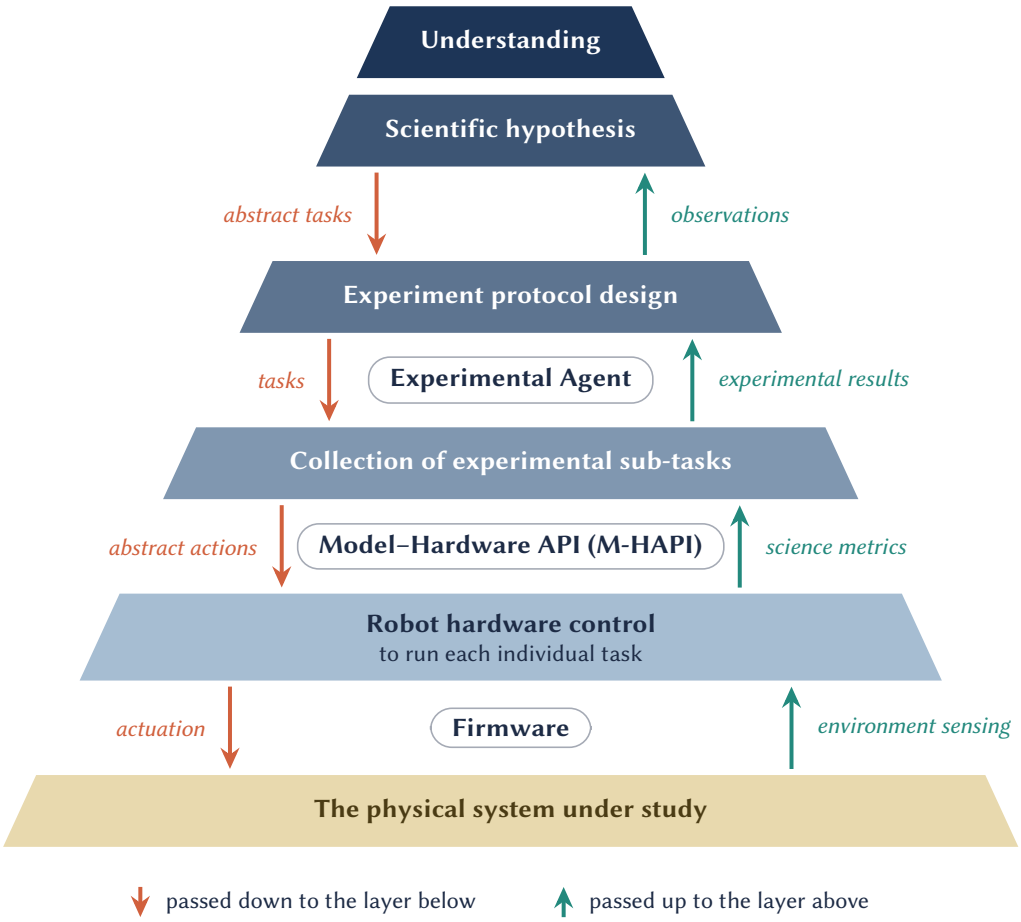


Figure 2. The hierarchy of a scientific experiment, from the physical system under study up to understanding. Each arrow shows what one layer passes to the next level; the boxes name the component that works at each interface in this project.

An agent performing at any of the levels needs specific information about the layers above and below it, but not all of it. Orchestration of activities across the layers is also important. The following are my design decisions for building this robotic experimentation system:

1. **Build a custom AI agent harness as a Python backend.** It holds the context about the system setup and tools as the system prompt, and the experiment protocol as the first user prompt. The harness calls the model API to decide which tools to call and how to present experiment results.

2. **Build a Model–Hardware API (M-HAPI) protocol:** an interface between the robotic control and the experiment sub-tasks. The need for this abstraction is quite clear, given that the robotic hardware interacts with the world through its own firmware and exposes only basic, fundamental action blocks

(move the nozzle to an x, y, z position). This is too low-level for the experimental sub-tasks. The recently announced Model Hardware Standard (not yet released at the time of writing) aims at the same problem. For my Model–Hardware API, I prompted Claude Code to explore the documentation for the Bambu Lab 3D printer and for the camera connected to a Raspberry Pi, and to set up basic primitives such as `set_temperature()` or `get_well_opacity_measurement(well_label)`. Writing the spec documentation and exploring and fixing hardware issues took a significant amount of the time for this project. A reliable and well-designed Model–Hardware interface is crucial.

3. **Build a digital twin of the system** to test the harness and the M-HAPI together before running physical experiments. A well-designed and well-scoped digital twin can uncover mistakes and shortcomings before the hardware is used. This simulated environment interfaces with the same M-HAPI and tries to mimic as much physics as possible in the core simulation engine. You can see it working as an experiment dashboard at https://34-9-34-15.sslip.io/. This web app interfaces with the agent harness running in the background as well as with the simulation engine, and twins the entire system end to end.

In the video you will see the same dashboard connected to the robot hardware, showing the actual camera feeds and the experimental data coming from the rig.

2 Significance of this project and further work

What I showed here is a simple yet complete example of an end-to-end system running an AI agent on an experimentation platform and putting an LLM in the loop with the physical system. The significance of this work is this very abstraction, and how we put the LLM in the loop. Let me explain with a contrasting example. As I was experimenting with the M-HAPI, I asked Claude Code to control the setup directly and obtain the kinetic coefficient of protein aggregation (the rate of egg-white whitening). Claude Code wrote a script to run this experiment loop, and the robot executed it with no issues. I later realized the egg white had not completely filled the well, so the opacity measurements would be off. This mistake would not have been discovered had I not looked at the wells manually. This is because the LLM's reasoning was not inside the loop for every frame. Claude did not look at every well as it was conducting the experiment; with the wrong abstraction, it outsourced the loop to an automated script. This is the same pattern used in a coding environment, and the model is post-trained for it very well. But the coding environment does not have the variability that exists in the physical world.

In contrast, while I was conducting the same experiment with the custom harness, the agent asked me to fill four wells, and I forgot to fill two. Because the LLM had to take a photo of each well and get the opacity number through an M-HAPI tool call, rather than write code, the model was induced to reason about each image and to stay constantly inside the experimental observations, at the obvious cost of more test-time compute. In this run, the agent notified me there was something wrong with the two wells, and I had to instruct it to ignore and run the experiment with $n = 2$.

This brings us to the first immediate direction for further work: what is the optimal level of abstraction across the six levels of the pyramid in Figure 2? I am actually not satisfied with the current, hard-coded version of the M-HAPI. Reading the press release for Anthropic's Model Hardware Standard, it appears that an AI agent interviews the scientist on how to set things up, but this is a one-time process. A better version would be an adaptive system that adjusts the boundary between agentic, LLM-reasoned work and hard-coded abstraction. Or I can imagine the Model–Hardware interface itself being run dynamically by a faster, cheaper LLM rather than by code, where the LLM produces code on the fly or uses some sort of domain-specific language to execute hardware commands. This would allow the harness to adapt to the complexity and noise of the physical system, and the trajectories of experimentation could be used

directly to post-train the LLM for scientific tasks. Learning from the Bitter Lesson, one should try to avoid hand-crafting abstractions between the levels, and instead let the model dynamically pay attention to the high- and low-level components of the experimental loop, using sub-agent processes. That is how a scientist moves between understanding at the high level and kneeling down at the instruments to fix the apparatus.

Another direction for further work is to incorporate safety mechanisms in the experimental loop. Currently, certain safety measures are included in the M-HAPI; for example, the bed temperature cannot be set above 100 °C. But thinking through every safety issue and hard-coding it into the API layers is neither the most flexible nor the most robust approach. I can imagine a separate, always-on agentic loop running across the abstraction layers and monitoring for various safety issues, guided by a broad system prompt rather than one that specifies rigid details. I wrote the spec for this but have not had time to implement and test it.

3 Time spent and logistics

The most time-consuming part of this project was building the physical setup: I had to retrofit the 3D printer, glue a camera to the filament nozzle, and test and calibrate the light and fixtures to be able to measure the opacity of egg white inside each well. The coding task itself was mostly writing the spec documents for the Model–Hardware API and the harness (3 hours). Once the spec and the setup were clean and accurate, Claude one-shotted most of the code. Testing took another 1–2 hours, and writing this document another 1.5 hours. I have yet to edit the video I took during the experiment, which should take about 30 minutes.

4 Final words

This was a very fun project. I am a theoretical and computational scientist, but I love working and dealing with physical labs and instruments. This project brought me back to the time I was a teenager doing DIY projects, something I have picked up again now, playing with my kids.

It was beautiful and magical to see Claude controlling the printer, looking through the camera and reasoning. In one session it saw me in the printer's side camera and recognized that it must be me! We are at the onset of another Claude Code moment, but for complete scientific exploration.

As we develop more capable models and harnesses for science, safety issues across the lab and model execution, as well as the broader use of AI tools for science at a high level, become even more important. We should take on these concerns at the same time as the tooling and the post-training of the model are being developed, rather than as an afterthought at deployment time.